

DS 6021: Intro to Predictive Modeling
Lab 4: Intro to sklearn + Feature Transformations

September 25, 2026
Due: September 29, 2026 11:59pm

Materials Required

All available in our class github repository:

- penguins.csv – Palmer penguins dataset
- bike-sharing-daily.csv – Daily bike rental counts, Washington DC, 2011–2012
- In-class notebooks and previous Lab Solutions

Activity Overview

So far this semester we have relied heavily on the statsmodels package. statsmodels is built for inference: it gives us standard errors, p-values, confidence intervals, and nested F-tests, and we have used all of them. Moving forward, we will almost always use scikit-learn (sklearn) much more, because it is built for prediction: preprocessing data, fitting models, and evaluating how they do on data they have never seen. Neither package is “better.” They are designed to answer different questions, and they are typically used in different contexts. For those of you who are thinking about ML Engineering or engineering adjacent roles, learning sklearn is a must.

This lab has three parts. In Part 1, you will rebuild a model from Lab 3 in sklearn and look closely at how sklearn handles categorical variables, including the difference between one-hot encoding and dummy variables. We covered this briefly in class, and this will allow us to dig in more fully.

In Part 2, we will spend a bit more time on variable transformation, specifically looking at scaling. This helps set the stage for recursive feature elimination with cross-validation (RFECV), which we will cover next week.

In Part 3, you will provide an update on the status of your final project. Although there is nothing formally due, I wanted to check in about this based on some of the feedback you provided earlier this week. You should not be saving this until the last minute, so I figured that now would be a good time to check in.

When answering the penguin questions in this lab, drop rows missing any of bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g, or sex, as we did in last week’s lab. You should have 333 penguins for each analysis.

Unless a question explicitly says otherwise, fit every model in this lab with sklearn.

Part 1: From statsmodels to scikit-learn (15 pts)

In this portion of the lab, you will fit the following additive model $\text{body_mass_g} \sim \text{flipper_length_mm} + C(\text{sex}) + C(\text{species})$ in both statsmodels and sklearn, and use it to look more carefully at one-hot encoding and dummy variables (Week 4, slides 18–22). The goal of this exercise is to get more familiar with scikit-learn and one-hot encoding.

Helpful references: [LinearRegression](#), [OneHotEncoder](#), [pd.get_dummies](#), and [Pipelines and ColumnTransformer](#).

1. **[2 pts]** Fit `body_mass_g ~ flipper_length_mm + C(sex) + C(species)` with `statsmodels`, as we have been doing in class. Report the coefficients and R^2 . What is the reference level for sex and for species? Interpret the coefficient on `C(species)[T.Gentoo]` in one sentence.
2. **[3 pts]** Now fit the same model with `sklearn`'s `LinearRegression`.
 - a. `sklearn` does not accept formulas, so you need to build the predictor matrix `X` and the response `y` yourself. Create the dummy variables with `pd.get_dummies(..., drop_first=True)` and fit the model. Then, look at `intercept_`, `coef_` for each coefficient, and R^2 using `.score()`. Confirm that they match your results from question 1.
 - b. Unlike `statsmodels`, `sklearn` does not calculate confidence intervals for the coefficients. How could we get these confidence intervals if we wanted them?
3. **[6 pts]** In Week 4 (slide 22), we said that one-hot encoding and dummy variables are the same thing, and that the difference is a convention as to what is kept and what is dropped. We touched upon this briefly in class. Now we will explore this in more detail.
 - a. Re-create `X` using `pd.get_dummies()` without `drop_first` (this matches `OneHotEncoder()` with its default settings). How many columns do you now have for sex, and for species?
 - b. Fit `LinearRegression()` with its default `fit_intercept=True`. Compare R^2 and the predicted values to question 2. Are they the same?
 - c. Compare the coefficients. Are they the same? Now calculate the differences between the Chinstrap and Adelie coefficients, the Gentoo and Adelie coefficients, and the male and female coefficients, and compare them to the coefficients we saw when we used dummy variables. What do you notice?
 - d. Explain what is going on. Why do you need to be careful about encoding?
4. **[4 pts]** In practice, `sklearn` users usually do not call `pd.get_dummies()` by hand. Instead, they bundle the encoding and the model together in a pipeline so the exact same steps are applied to any new data being used for predictions. Using the `sklearn Pipeline` functionality is an important conceptual and practical step. Model development pipelines, whether in `sklearn` or another framework, are a critical part of the practice of data science.
 - a. Build a `Pipeline` that uses a `ColumnTransformer` to pass `flipper_length_mm` through unchanged and apply `OneHotEncoder(drop="first")` to sex and species, followed by `LinearRegression()`. Fit it on the penguin `DataFrame` directly. Use `get_feature_names_out()` on the `ColumnTransformer` to confirm that the columns and coefficients are the same as in Question 2.
 - b. Using this model, predict body mass for a male Gentoo penguin with a flipper length of 215 mm, and verify this prediction by hand using your coefficients from Question 1. You can use your coefficients from either the `statsmodels` or `sklearn` model.

Part 2: Scaling and Variable Transformations (18 pts)

In Lab 2, you used planet data and a log transformation to derive Kepler's law. In Lab 3, you read more about feature transformations. In this exercise, we will explore transformations a bit more and look at how transforming data changes the model. There are two different kinds of transformations we will explore. The first is linear transformations (changing units, centering, standardizing, min-max scaling). These transformations linearly stretch and shift a variable without changing its shape. Nonlinear transformations, like the log transformation we used for the planets data in Lab 2, change the shape of the

data. These have very different effects on a linear model, and one of them is essential for recursive feature elimination (RFE).

For questions 1–3, use the 333 penguins from Part 1 and the model `body_mass_g ~ flipper_length_mm + bill_length_mm + bill_depth_mm`.

Helpful references: the sklearn [preprocessing guide](#) and the UVA Library article on [interpreting log transformations](#) from the Lab 3 readings.

1. **[2 pts]** Fit the model with sklearn using the original units, and report the coefficients and R^2 .
 - a. RFE works by fitting the model, ranking the features by the absolute value of their coefficients, and dropping the smallest one. Based on your coefficients, which feature would RFE drop first?
 - b. Now convert `bill_length_mm` to centimeters (divide by 10) and refit. What happens to the coefficients, R^2 , and predictions? Which feature would RFE drop first now?
2. **[5 pts]** Now standardize the three predictors and refit, using `make_pipeline(StandardScaler(), LinearRegression())`.
 - a. Report the coefficients, and interpret the coefficient on flipper length in words. (Hint: what is “one unit” of a standardized variable?)
 - b. Rank the features by the absolute value of their standardized coefficients. Which would RFE drop first? Repeat with bill length in centimeters. Does the unit conversion matter now? Why or why not?
 - c. Repeat the model fitting but replace `StandardScaler` with `MinMaxScaler` and `RobustScaler` instead of `StandardScaler`. Do the three scalers agree on the ranking? On the magnitudes? What is each scaler using as its “unit”?
 - d. Repeat the model fitting, but instead of using a Scaler, center the predictors (subtract the mean, but do not divide by the standard deviation)
3. **[6 pts]** Build a table with one row for each of the following versions of the model: (i) original units, (ii) bill length in cm, (iii) centered predictors, (iv) `StandardScaler` and (v) `MinMaxScaler`. The columns should include: coefficients, intercept, R^2 .
 - a. Add three columns to the table that indicate whether each of the following changed relative to (i): the coefficients, the intercept, R^2 , and the predicted values. The first row should be False (since comparing a model with itself should always result in no change!)
 - b. Fit (i) and (iv) in statsmodels and compare the t-statistics and p-values for the slopes. What do you notice? What does this tell you about how linear scaling impacts model coefficients and predictions.
 - c. Rebuild the original model, but this time, take the log of the predictors. Add this row to your table above. Comment on differences you observe between the linear transformations and this one.
4. **[5 pts]** Now we will switch to the bike data and look at the number of casual riders to further explore non-linear transformations.
 - a. Plot a histogram of count of casual riders. On the x-axis should be the count of casual riders, and the y-axis is the number of days with that number of riders. Comment on the shape of this plot.
 - b. Fit `casual ~ temp + hum + windspeed + workingday` and plot fitted values vs. residuals. What do you see?

- c. Fit the same model with $\log(\text{casual})$ as the response. (Check whether casual contains any zeros first. If it does, use `np.log1p`.) Plot the histogram and fitted vs. residuals again. What changed?
- d. Based on our readings from last week, did it make sense to do the log transformation on the target variable, based on the original histogram shape? Based on the plots in part c, did this have the impact you expected? How does this compare with what we did previously for Kepler's Law?

Part 3: Final Project Update (7 pts)

During the first week of class, I provided rubric and details about our [Final Project](#). I would like to use part of this assignment as a check in about this project. Please complete the following:

- Let me know who you are working with for this project. The rubric suggests groups of 3-4, but 5 is fine as well. Please provide a list of individuals on your team, along with computing IDs.
- Share the problem space you have selected, data you plan to use, and problem you intend to solve. This should be a paragraph.
- [Optional] If you have already started work beyond this, please provide a few sentences letting me know where you are at.

Note: You can work together on this Part of the Lab Activity, and only one member of your group needs to turn it in. Please indicate who is turning in this portion of the Lab and where to find it, and you will all get full credit. You can upload a document to Canvas or add to github as a separate file, but please do not include this in your notebook for Parts 1+2 of the lab.

Deliverables

For Part 1+2, please include the following:

- All code used to complete the activities. This can be turned in as a single Jupyter notebook, multiple Jupyter notebooks, or one or more .py files. Up to you!
- For open ended questions, please include a few sentences or a paragraph answering the question. This text can be included in your Jupyter notebook as a markdown cell or a separate document.
- Per our policy on Generative AI detailed in the syllabus, if you use it to generate code, you need to flag how and where you used it, and how you ascertained the correctness of what you did. You are not allowed to use Generative AI for any of the written explanations or reflection questions. If you did not use Generative AI for any part of the assignment, please state that explicitly.